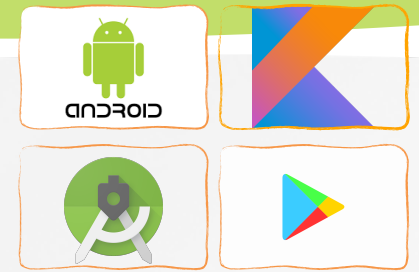




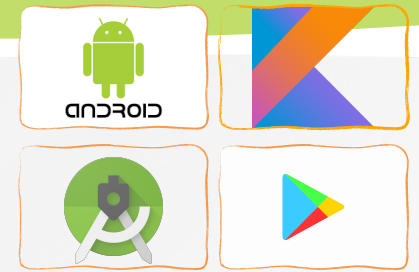
Software Development for Mobile Devices

Nguyen Le Hoang Dung
[*nlhdung@fit.hcmus.edu.vn*](mailto:nlhdung@fit.hcmus.edu.vn)



Realm

- » Realm is a mobile database which is an open source, developer-friendly alternative to CoreData and SQLite.
 - It's available for Kotlin, Swift, Objective-C, Java, C#, and JavaScript
- » Why Use Realm
 - Intuitive to Developers: Realm's object-oriented data model is simple to learn, doesn't need an ORM, and lets you write less code.
 - Designed for Offline Use: Realm's local database persists data on-disk, so apps work as well offline as they do online.
 - Built for Mobile: Realm is fully-featured, lightweight, and efficiently uses memory, disk space, and battery life.
 - ...



Example 1

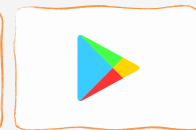
Installation

» Add the Gradle dependencies

- Add `io.realm.kotlin` in your **project-level Gradle build file**, typically found at **<project>/build.gradle**:

```
// Top-level build file where you can add configuration
options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false

    id("io.realm.kotlin") version "1.16.0" apply false
}
```



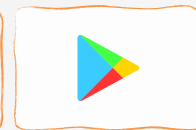
Example 1

Installation

» Add the Gradle dependencies

- Add the following to your **app-level Gradle build file**, typically found at **<project>/app/build.gradle**:

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
    id("io.realm.kotlin")  
}  
android {  
    namespace = "com.hd.realmdemo"  
    compileSdk = 34  
    defaultConfig { ...  
        applicationId = "com.hd.realmdemo"  
    }  
    buildTypes { ...  
    }  
    ...  
}  
...
```



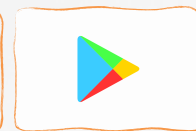
Example 1

Installation

» Add the Gradle dependencies

- Add the following to your **app-level Gradle build file**, typically found at **<project>/app/build.gradle**:

```
...  
dependencies {  
  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    // other libs ...  
  
    implementation("io.realm.kotlin:library-base:1.16.0")  
    // If using Device Sync  
    implementation("io.realm.kotlin:library-sync:1.16.0")  
    // If using coroutines with the SDK  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.0")  
}
```



Example 1

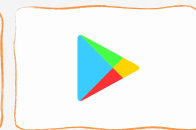
Implementation of Realm

» Create model class

```
import io.realm.RealmObject

class Person : RealmObject {
    var name: String = "Foo"
    var dog: Dog? = null
}

class Dog : RealmObject {
    var name: String = ""
    var age: Int = 0
}
```



Example 1

Implementation of Realm

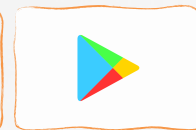
» Open Database

```
val REALM_NAME = "HD_REALM_NAME"
lateinit var realm : Realm

//Open Database
fun openDatabase(){
    val configuration = RealmConfiguration.Builder(
        setOf(Person::class, Dog::class))
        .name(REALM_NAME).build()
    // specify name so realm doesn't just use
    // the "default.realm" file

    realm = Realm.open(configuration)
    Log.i("hdlog", realm.configuration.path)
    ///data/user/0/com.nlhdung.realmapp/files/HD_REALM_NAME

    writPerson()
}
```

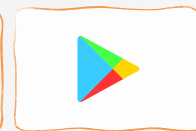


Example 1

Implementation of Realm

» Write

```
fun writPerson() {  
    // plain old kotlin object  
    val person = Person().apply {  
        name = "Dung Nguyen"  
        dog = Dog().apply { name = "Fido"; age = 2 }  
    }  
  
    // persist it in a transaction  
    realm.writeBlocking {  
        val managedPerson = this.copyToRealm(person)  
        Log.i("hdlog", "Wrote " + managedPerson.name)  
    }  
}
```

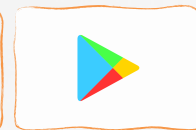



Example 1

Implementation of Realm

» Find all objects of a type as a results collection, synchronously

```
fun readAllWithName(name: String){  
    val all = realm.query<Person>().find()  
    Log.i("hdlog", "Read " + all.size + " elements")  
  
    // Persons named 'Dung'  
    //val personsByNameQuery = realm.query<Person>("name = $0", name)  
    val personsByNameQuery = realm.query<Person>("name contains $0",  
        name)  
    val filteredPersons = personsByNameQuery.find()  
    Log.i("hdlog", "Read " + filteredPersons.toString())  
}
```



Example 1

Implementation of Realm

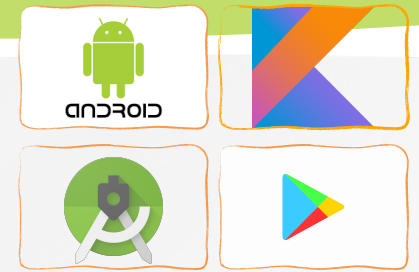
- Finds all objects that fulfill the query conditions and returns them asynchronously as a Flow

```
fun readAsync(){
    val personsByNameQuery = realm.query<Person>("name contains $0", "Dung")

    val pObserver = GlobalScope.async {
        personsByNameQuery.asFlow().collect { results ->

            when (results) {
                is InitialResults<Person> -> {
                    for (p in results.list) {
                        Log.i("hdlog", "Info async:" + p.toString())
                    }
                }
                else -> { /* no-op */ }
            }
        }
    }

    // ... Later, cancel the Flow, so you can safely close the database
    //pObserver.cancel()
    //realm.close()
}
```

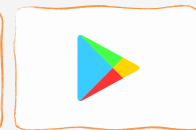


Example 1

Implementation of Realm

– Update

```
fun readAndUpdate(term: String, newName: String){  
    val personsByNameQuery = realm.query<Person>("name contains $0",  
        term)  
    val filteredPerson = personsByNameQuery.find().first()  
  
    realm.writeBlocking{  
        findLatest(filteredPerson)?.name = newName  
    }  
}
```

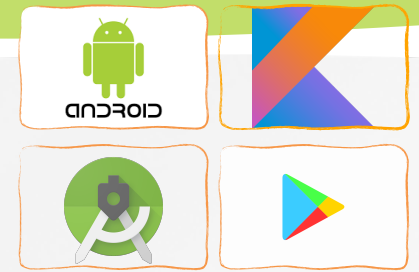


Example 1

Implementation of Realm

– Delete

```
fun deletePersons() {  
    realm.writeBlocking {  
        val results = realm.query<Person>().find()  
        results.forEach { delete(findLatest(it)!!) }  
    }  
}
```

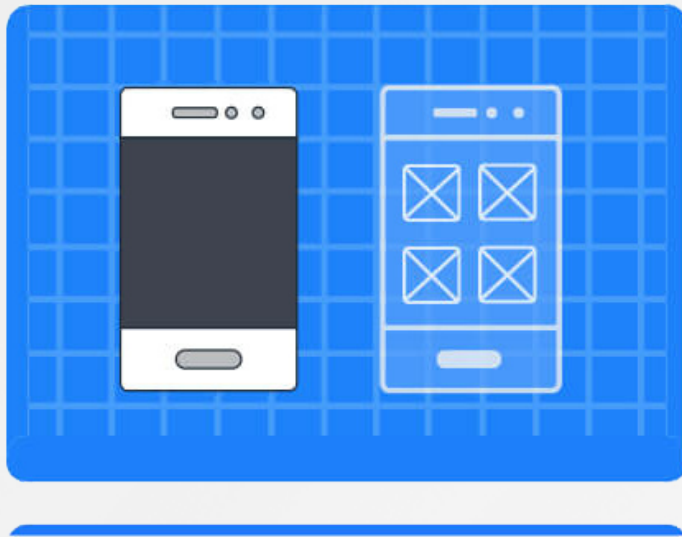
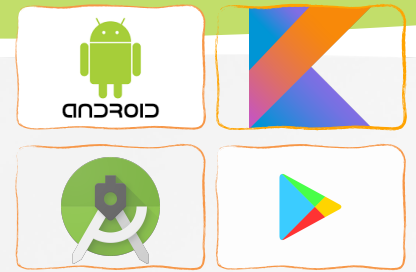


Example 1

Implementation of Realm

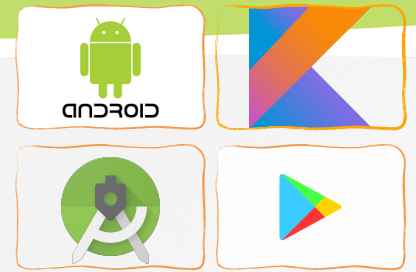
- » Reading about observing data changes
 - Realm support asynchronous observers on all its data structures
 - Realm data
 - RealmObject
 - RealmLists
 - ...

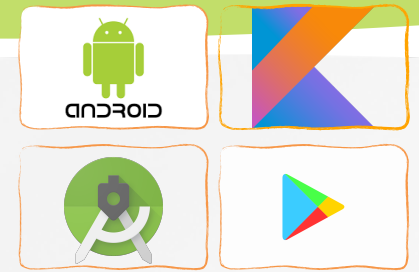
Demo



 **Kotlin**

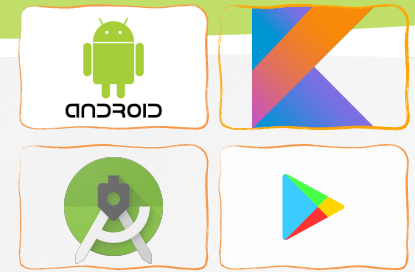
Q&A





Practice

- Student management application for managing students using Realm database
 - Add student (id, name, birthday, point...)
 - Display list of students
 - Select and update a student



Reference

- » <https://kotlinlang.org/docs/reference/>
- » Peter Späth, Learn Kotlin for Android Development, 2019
- » Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- » Peter Späth, Pro Android with Kotlin, 2019
- » Milos Vasic, Mastering Android Development with Kotlin, 2017
- » <https://github.com/realm/realm-kotlin>
- » <https://docs.mongodb.com/realm/sdk/kotlin/#local-realm-database>
- » <https://docs.mongodb.com/realm/sdk/kotlin/realm-database/query-language/>